

Context rot: the emerging challenge that could hold back LLM progress

What if attention isn't all you need?



TIMOTHY B. LEE

NOV 10, 2025 • PAID



90



5

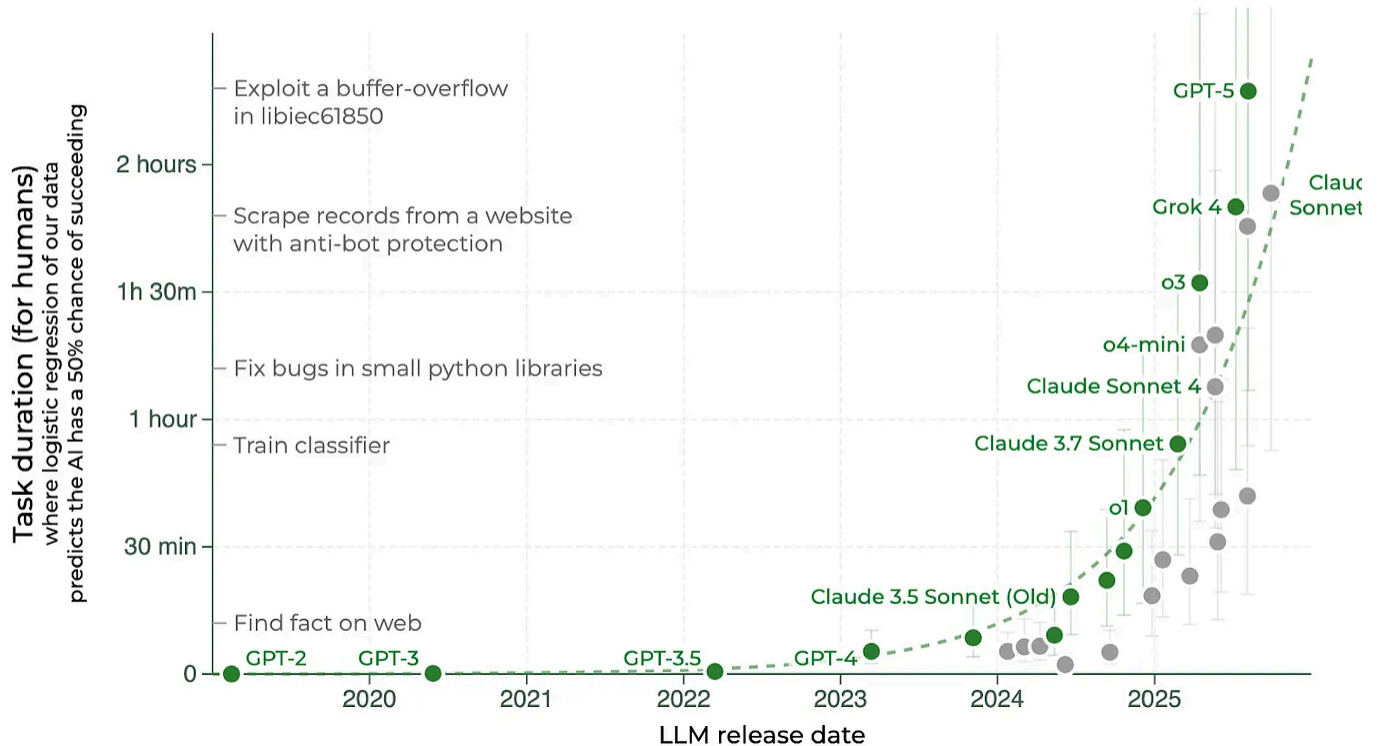


8

Sh

Many people believe that the next frontier for large language models is task length. [March study](#) from the research organization METR documented that large language models have steadily gotten better at performing software engineering tasks that require significant time when performed by a human being. If anything, progress seems to be accelerating this year. Here's an updated version of their chart:

Time-horizon of software engineering tasks different LLMs can complete 50% of the time



If this trend continues, in a few years LLMs will be able to complete tasks that take human programmers multiple days. Maybe a few years after that it'll be weeks, and then months. If the trend continues long enough, we could wind up with models that can take over large-scale software engineering projects, putting many human programmers out of work and further accelerating AI progress.

I don't doubt that the trend toward longer task lengths still has some room to run. I suspect that relatively soon, we're going to bump up against fundamental limitations of the attention mechanism underlying today's leading LLMs.

With attention, an LLM effectively "thinks about" every token in its context window before generating a new token. That works fine when there are only a few thousand tokens in the context window. But it gets more and more unwieldy as the number of tokens grows into the hundreds of thousands, millions, and beyond.

An analogy to the human brain helps to illustrate the problem. As I sit here writing this article, I'm *not* thinking about what I ate for breakfast in 2019, the acrimonious breakup I had in 2002, or the many episodes of *Star Trek* I watched in the 1990s. If my brain were constantly thinking about these and thousands of other random topics, I'd be too distracted to write a coherent essay.

But LLMs *do* get distracted as more tokens are added to their context window — a phenomenon that has been dubbed “context rot.” Anthropic researchers explained it in a [September blog post](#):

Context must be treated as a finite resource with diminishing marginal returns. Like humans, who have limited working memory capacity, LLMs have an “attention budget” that they draw on when parsing large volumes of context. Every new token introduced depletes this budget by some amount, increasing the need to carefully curate the tokens available to the LLM.

This attention scarcity stems from architectural constraints of LLMs. LLMs are based on the transformer architecture, which enables every token to attend to every other token across the entire context. As its context length increases, a model's ability to capture these pairwise relationships gets stretched thin, creating a natural tension between context size and attention focus.

The blog post went on to discuss context engineering, a suite of emerging techniques for helping LLMs stay focused by removing extraneous tokens from their context windows.

Those techniques are fine as far as they go. But I suspect they can only mitigate the underlying problem. If we want LLMs to reason effectively over much longer contexts, we may have to fundamentally rethink how LLMs work.

Structure matters

In college I paid my rent by working as a web programmer for the University of Minnesota. One of my first projects was to build a simple web application powered by a [relational database](#). It worked fine in testing, but it became glacially slow with real user data. I didn't understand why.

When I asked a more experienced programmer about it, his first question was “did you add an index to the database?”

“What's an index?” I asked.

I soon learned that a database index works a lot like the index of a book.

Suppose you're trying to find the first page in a history book that mentions Abraham Lincoln. If the book has no index, you'll have to scan every page. This might take several minutes if it's a long book. But if there is an index, its alphabetical structure will allow you to find the right page in a few seconds.

A database index has the same basic function: organize information so it's easy to find. As I learned the hard way, an index becomes more and more necessary as data is added to a database.

This kind of scaling analysis is fundamental to any computer science curriculum. As a computer science major, I learned how to determine whether a computer program would scale gracefully or — like my database with no index — choke when applied to large data sets.

So when I started to study [how large language models work](#), I was shocked to learn that one of the foundational concepts, the attention mechanism, has terrible scaling properties. Before an LLM generates a new token, it compares the most recent token to every previous token in its context window. This means that an LLM consumes more and more computing power — per token — as its context window grows.

If there are 101 previous tokens, it takes 100 attention operations to generate the next token. If there are 1,001 previous tokens, it takes 1,000 attention operations. And the costs are *per token*, so a session with 10 times more tokens takes about 100 times more computing power.¹

Good programmers try to avoid using algorithms like this. Unfortunately, nobody has found a viable alternative to attention.

So AI companies have tried to overcome the problem with engineering muscle instead. They've developed clever algorithms like [FlashAttention](#) that minimize the computational cost of each attention operation. And they've built massive data centers optimized for attention calculations. For a while, these efforts had impressive results: context windows grew from 4,096 tokens in 2022 to a [million tokens](#) in early 2024.

Industry leaders hope to continue this trend with even more engineering muscle. In a July interview with Alex Kantrowitz, [Anthropic CEO Dario Amodei said](#) that “there's no reason we can't make the context length 100 million words today, which is roughly what a human hears in their lifetime.”

I don't doubt that Anthropic could build an LLM with a context window of 100 million tokens if it really wanted to — though using it might be stupendously expensive. But I don't think anyone will be happy stopping at 100 million tokens.

For one thing, that 100 million figure seems like an underestimate for the number of tokens humans “process” over a lifetime. Studies show the average adult speaks [around 15,000 words per day](#) — which works out to around 400 million words over a lifetime. Presumably, most people hear a similar number of words, and read a lot of words as well. They also experience a lot of images, sounds, smells, and other sensations. If we represent all of those experiences as tokens, I bet the total would comfortably exceed 1 billion.

Moreover, AI companies aren't just trying to match human performance, they're trying to dramatically exceed it. That could easily require models to process a lot more.

More context, more problems

But there's also a deeper problem. Today's leading LLMs don't effectively use the million-token context windows they already have. Their performance predictably degrades as more information is included in the context window.

In November 2023, OpenAI released GPT-4 Turbo, the first model with 128,000 tokens of context. Later that same month, Anthropic released Claude 2.1, the first model with 200,000 tokens of context.

Greg Kamradt was one of the first people to [perform](#) a [needle-in-a-haystack test](#) on these models. He took a long document and randomly inserted a “needle” sentence like “The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day.”

Then he'd ask an LLM “what is the best thing to do in San Francisco?” and see if it could answer. He found that both GPT-4 Turbo and Claude 2.1 performed worse on this task as the context length increased — especially if the “needle” was in the middle of the document:

Frontier labs worked hard to improve performance on this kind of task. By the time Anthropic released Claude 3 in March 2024, needle-in-a-haystack performance was [lot better](#). But this is the simplest possible test of long-context performance. What about harder problems?

In February 2025, a team of researchers at Adobe [published research](#) on a more difficult variant of the needle-in-a-haystack test. Here the “needle” was a sentence | “Yuki lives next to the Semper Opera House,” and the model would be asked “Which character has been to Dresden?”

To answer this question, you need to know that the Semper Opera House is in Dresden. Leading language models *do* know this, so if you give them this challenge a short prompt (a small “haystack”) they tend to get it right more than 90% of the time. But if you give them the same challenge in a larger “haystack” — for example, a 32,000 token prompt — accuracy drops dramatically:

- GPT-4o goes from 99% to 70%

- Claude 3.5 Sonnet goes from 88% to 30%
- Gemini 2.5 Flash goes from 94% to 48%
- Llama 4 Scout goes from 82% to 22%

Long-context performance dropped even further when the researchers asked “which character has been to the state of Saxony.” This question required the model to recognize that the Semper Opera House is in Dresden *and* that Dresden is in Saxony. The longer the context got, the worse models tended to do on questions like this that required two reasoning “hops”:

So not only do LLMs perform worse as more tokens are added to their context, they exhibit more severe performance degradation on more complex tasks.² I think this bodes poorly for getting LLMs to do the kind of work that takes human workers days, weeks, or even months. These tasks will not only require a lot of tokens, they're also far more complex than contrived needle-in-a-haystack benchmarks.

The curse of context rot

Photo by Serhii Luzhevskyi via iStock Editorial / Getty Images Plus.

And indeed, technologists have noticed that LLM performance on real-world tasks tends to decline as contexts get longer.

In June, a Hacker News commenter [coined the phrase](#) “context rot” to describe the phenomenon where LLMs become less effective as the size of their context grows.’

startup Chroma [published a widely read study](#) on the phenomenon in July.

[Click here](#) for a 15 percent discount on an annual subscription!

No one fully understands how LLMs work, so it's hard to say exactly why context rc happens. But here's how I think about it.

Keep reading with a 7-day free trial

Subscribe to ***Understanding AI*** to keep reading this post and get 7 days of free access to the full post archives.

Start trial

Already a paid subscriber? [Sign in](#)